

CARdex: Um Sistema Inteligente para Identificação, Coleção e Gamificação de Veículos Automotivos

Ana Fernanda Souza Cancado, Arthur de Sá Braz de Matos,
Gabriel Praes Bernardes Nunes, Guilherme Otávio de Oliveira

¹Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brazil

{afscancado, amatos, gabriel.nunes.1205431, gooliveira}@sga.pucminas.br

Abstract. *Car spotting is a popular activity among car enthusiasts, where users photograph and share images of rare or interesting vehicles. However, current interactions occur mainly through unstructured social media posts, lacking gamification, organization, or dedicated community features. This work presents CARdex, an intelligent system that applies computer vision to automatically identify vehicle make and model, enabling users to build personalized digital collections, compare findings, and engage socially. The system integrates an EfficientNet-B3 model trained on 742 automotive classes, a distributed architecture involving React, Render, Cloudflare Tunnel, and Google Colab, and a database for user and collection management. Preliminary results show a validation accuracy of 89.76% and reliable distributed communication among system components.*

Resumo. *O car spotting é uma prática popular entre entusiastas de automóveis, que consiste em registrar e compartilhar fotos de veículos considerados raros, exclusivos ou interessantes. Entretanto, a experiência atual é fragmentada em redes sociais, sem mecanismos de organização, gamificação ou integração social estruturada. Este trabalho apresenta o CARdex, um sistema inteligente que utiliza técnicas de visão computacional para identificar automaticamente marca e modelo de veículos, permitindo ao usuário montar uma coleção digital própria, comparar registros e interagir com outros entusiastas. O sistema combina um modelo EfficientNet-B3 treinado em 742 classes automotivas, uma arquitetura distribuída composta por React, Render, Cloudflare Tunnel e Google Colab, além de um banco de dados para gerenciamento de usuários e coleções. Resultados preliminares indicam acurácia de 89.76% no conjunto de validação e funcionamento consistente da comunicação distribuída entre os módulos.*

1. Introdução

A comunidade de car spotters é composta por entusiastas que registram e compartilham imagens de veículos considerados interessantes, raros ou exclusivos. Apesar de numerosa e ativa, essa comunidade carece de uma plataforma dedicada que ofereça organização, gamificação e interação social estruturada. Atualmente, a experiência é limitada a postagens dispersas em redes sociais, dificultando a construção de coleções pessoais, a comparação de registros e o engajamento contínuo.

Com o avanço de técnicas de visão computacional e aprendizado profundo, torna-se possível automatizar a identificação de veículos, permitindo experiências mais ricas e gamificadas. Nesse contexto, propõe-se o *CARDex*, um sistema que combina reconhecimento automático de marca e modelo de carros com elementos de coleção digital e interação social.

O objetivo principal do sistema é oferecer uma experiência integrada em que usuários possam registrar veículos, montar coleções personalizadas, interagir com outros participantes e explorar funcionalidades de gamificação que incentivem a descoberta de novos modelos.

2. Trabalhos Relacionados

O reconhecimento automático de veículos, conhecido como *Vehicle Make and Model Recognition* (VMMR), tem avançado significativamente com o uso de técnicas de *deep learning*. Métodos modernos buscam lidar com a alta variabilidade visual entre marcas, modelos e versões de automóveis. Entre eles, destaca-se o modelo *Two-Branch-Two-Stage*, proposto por [Lyu et al. 2022], no qual o processo de decisão é dividido em dois ramos independentes: um para a predição da marca e outro para a predição do modelo. Essa arquitetura reduz ambiguidades e melhora o desempenho em ambientes com grande número de classes.

Outro campo relevante é o *Vehicle Re-Identification* (Re-ID), cujo objetivo é rastrear um mesmo veículo através de múltiplas câmeras distribuídas em ambientes urbanos. De acordo com [Zheng et al. 2024], essa tarefa enfrenta desafios como mudanças de ângulo, iluminação variável e oclusões parciais, exigindo modelos robustos capazes de capturar características discriminativas mesmo em condições adversas.

No contexto comercial, aplicativos como o CarGPT utilizam modelos de visão computacional para identificar veículos e fornecer informações técnicas como preço médio e ano de fabricação, demonstrando a aplicabilidade prática desses sistemas.

Além disso, abordagens baseadas em gamificação, como observadas em jogos do tipo Pokémon Go, destacam o forte potencial de engajamento quando o usuário é incentivado a colecionar elementos do mundo real. Essa linha inspira soluções que combinam identificação automática com mecânicas de coleta e progressão, tornando a interação mais atrativa.

3. Metodologia

A metodologia adotada para o desenvolvimento do *CARDex* foi estruturada em uma arquitetura distribuída que combina componentes de front-end, back-end, computação em nuvem e serviços especializados de inteligência artificial. O objetivo dessa abordagem é garantir escalabilidade, modularidade e eficiência no processamento das imagens enviadas pelos usuários, mantendo a experiência fluida e responsiva.

3.1. Arquitetura Geral do Sistema

O sistema é composto por quatro camadas principais: (i) front-end desenvolvido em React, responsável pela interação com o usuário; (ii) um gateway/API hospedado no Render, que atua como intermediário de comunicação; (iii) um back-end de IA executado no

Google Colab e exposto via Cloudflare Tunnel; e (iv) um banco de dados para armazenamento estruturado das informações de usuários, veículos registrados e interações sociais.

A comunicação entre as camadas ocorre por meio de requisições HTTP, transmitindo imagens e recebendo previsões de marca e modelo do veículo identificado pela IA.

3.2. Front-end em React

O front-end foi desenvolvido utilizando o framework React, adotado por sua modularidade, rapidez e facilidade de integração com APIs externas. Essa interface permite:

- Upload da imagem do veículo.
- Visualização dos resultados da classificação.
- Gestão da coleção pessoal de veículos identificados.
- Navegação entre áreas do sistema: permitir que os usuários transitem entre diferentes seções, incluindo a comparação de coleções e a visualização de perfis. Essas funcionalidades serão consideradas como trabalhos futuros.

A interface prioriza simplicidade, responsividade e clareza, garantindo uma experiência agradável mesmo em redes de conexão limitada.

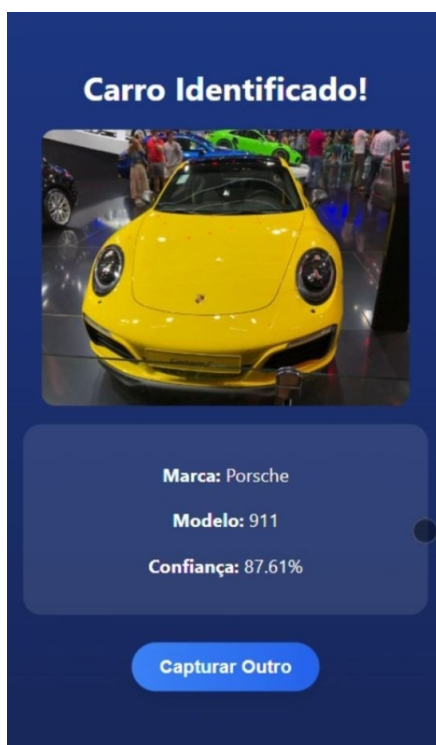


Figure 1. Front-end identificando carro

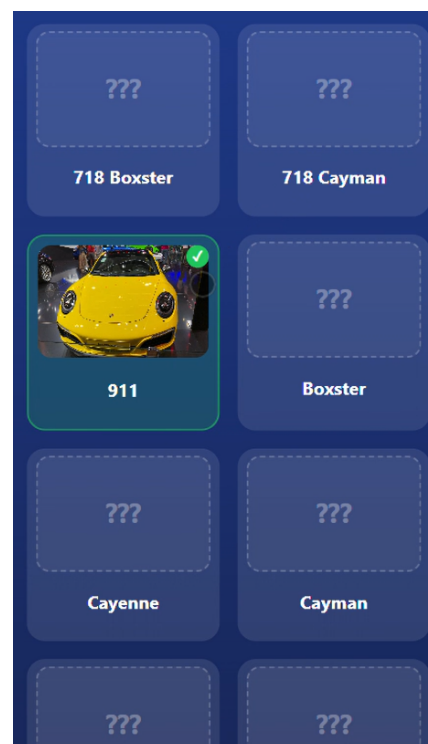


Figure 2. Coleção digital de carros capturados

3.3. Implementação da Inteligência Artificial - Processamento de Imagens

O desenvolvimento do componente de inteligência artificial deste trabalho fundamentou-se na metodologia e no dataset proposto por [Huang et al. 2022] no projeto *DVM-CAR*. O conjunto de dados original foi adaptado para atender ao escopo deste projeto.

A estrutura hierárquica dos dados, organizada em *Marca/Modelo*, permitiu a implementação de uma arquitetura de classificação em cascata (*two-stage classification*). Esta abordagem divide o problema em etapas sequenciais, reduzindo a complexidade da classificação em um espaço de busca com muitas classes.

3.3.1. Ambiente Computacional e Dependências

O ambiente de desenvolvimento foi estruturado utilizando a linguagem Python como base, apoiada por bibliotecas voltadas para *Deep Learning* e processamento de imagens. A escolha do framework PyTorch [Paszke et al. 2019] deve-se à sua flexibilidade na prototipagem de arquiteturas neurais e suporte nativo à aceleração por hardware.

As principais ferramentas utilizadas incluíram:

- **PyTorch e torchvision:** para construção, treinamento e avaliação das Redes Neurais Convolucionais (CNNs);
- **OpenCV e Pillow:** para pré-processamento, redimensionamento e normalização das imagens;
- **scikit-learn:** para computação de métricas estatísticas de desempenho;
- **FastAPI:** para encapsulamento do modelo final em uma interface de serviço (API).

O treinamento dos modelos foi realizado em ambiente com suporte a processamento em GPU (arquitetura CUDA), utilizando técnicas de precisão mista (*mixed precision*) para otimizar o uso de memória e reduzir o tempo computacional necessário para a convergência dos modelos.

3.3.2. Arquitetura em Dois Estágios: Classificação de Marca e Modelo

O processo de reconhecimento é estruturado em dois estágios sequenciais. No primeiro estágio, dedicado à identificação da fabricante do veículo, emprega-se a arquitetura *EfficientNet-B3* [Tan and Le 2019], escolhida por seu balanço eficiente entre profundidade, largura e resolução obtido por meio do método de *compound scaling*. A camada classificadora original foi substituída por uma cabeça totalmente conectada personalizada, ajustada para o número de marcas presentes no conjunto de dados.

No segundo estágio, a classificação do modelo é realizada por um conjunto de classificadores independentes baseados na arquitetura *EfficientNet-B0*. Para cada fabricante reconhecida no estágio anterior, existe um modelo especialista correspondente, treinado exclusivamente para distinguir entre os modelos daquela marca. Essa especialização permite capturar características discriminativas sutis entre modelos visualmente semelhantes pertencentes à mesma fabricante, aumentando a precisão e reduzindo ambiguidades.

3.3.3. Métricas de Avaliação

A validação dos modelos baseou-se principalmente na métrica de Acurácia, definida pela razão entre as predições corretas e o total de amostras avaliadas.

Adicionalmente, implementou-se o cálculo da confiança da predição, obtida através da função *Softmax* na camada de saída.

3.4. Arquitetura de Computação Distribuída

O sistema foi desenvolvido usando uma arquitetura distribuída, onde diferentes componentes especializados trabalham juntos para realizar tarefas específicas. Essa abordagem modular traz benefícios importantes: facilita a manutenção do código, permite que o sistema cresça conforme a demanda e melhora a segurança ao separar as responsabilidades em diferentes camadas.

3.4.1. Render: O Backend da Aplicação

O Render atua como servidor backend, fazendo a ponte entre a interface que o usuário vê e o serviço de inteligência artificial que processa as imagens. Ele funciona como um intermediário que organiza e controla o fluxo de informações.

Como receptor de requisições, o Render recebe todas as solicitações vindas do site, como quando o usuário faz upload de uma imagem. Manter essa separação entre a interface e o processamento é importante porque permite modificar uma parte sem afetar a outra.

Na função de roteador, o servidor direciona cada requisição para o modelo de IA rodando no Colab. O acesso ao modelo é feito através do *Cloudflare Tunnel*, que cria um endereço público seguro. Essa camada intermediária protege os detalhes técnicos do modelo, permitindo mudanças na infraestrutura sem quebrar a comunicação com o front-end.

Como gerenciador de dados, depois que o modelo retorna sua análise, o Render faz duas coisas importantes: salva as informações relevantes (a previsão e detalhes da imagem) no banco de dados e organiza a resposta antes de enviá-la de volta para o site, garantindo que o usuário receba um resultado bem formatado.

3.4.2. Google Colab: Onde o Modelo Roda

O Google Colab é o ambiente onde o modelo de aprendizado de máquina realmente funciona. Ele oferece poder computacional adequado para processar IA, incluindo acesso a GPUs que aceleram os cálculos.

Para hospedar o modelo, o Colab é configurado para receber requisições pela internet através de um túnel do *Cloudflare*. Essa solução cria um endereço público seguro que transforma o notebook em um serviço acessível pela web.

A análise da imagem acontece quando o Colab recebe a foto enviada pelo Render. O modelo gera suas previsões e o formato de resposta usado é o JSON.

3.4.3. Comunicação

A comunicação entre as partes do sistema acontece através de endpoints HTTP. O Colab tem seu endpoint (criado pelo *Cloudflare Tunnel*) que aceita requisições POST com imagens, enquanto o Render expõe sua própria API para o site React.

3.5. Estratégias de Paralelização

Para otimizar o desempenho do sistema de classificação de carros, foram implementadas duas estratégias complementares de paralelização: *batch processing* e *multi-threading*, visando maximizar a utilização da GPU NVIDIA GeForce RTX3060.

3.5.1. Batch Processing

O *batch processing* agrupa múltiplas imagens em um único lote e as processa simultaneamente na GPU. Em vez de N transferências individuais CPU $\rightarrow$ GPU, realiza-se apenas uma transferência do batch completo de dimensões $[N, 3, 336, 336]$, seguida de uma única passada pelo modelo (*forward pass*).

Table 1. Comparação: processamento individual vs. batch (8 imagens)

Métrica	Individual	Batch
Transferências CPU $\rightarrow$ GPU	8	1
Tempo total (ms)	272	41
Utilização da GPU	30-40%	80-90%
Speedup	1.0×	6.6×

O *batch processing* alcançou speedup de 6.6× (redução de 85% no tempo), principalmente pela redução do overhead de comunicação CPU-GPU e melhor aproveitamento do paralelismo de dados.

3.5.2. Multi-threading

Para requisições concorrentes de múltiplos usuários, implementou-se paralelização com `ThreadPoolExecutor`, distribuindo imagens entre threads que compartilham uma única instância do modelo *EfficientNet B3* em memória. Esta arquitetura elimina overhead de serialização e permite *pipelining*: enquanto uma thread aguarda a GPU, outras preparam dados.

4. Resultados

4.1. Escalabilidade Forte

Mantendo o problema fixo (135 imagens) e variando threads de 1 a 32, obteve-se:

Os resultados revelam três fases: (1) Ganhos significativos até 4 threads (speedup 2.77×, eficiência 69%); (2) Estagnação em 8 threads (speedup máximo 3.06×, eficiência cai para 38%) devido à saturação da GPU e contenção de VRAM; (3) Degradação com 32 threads (desempenho pior que 4 threads), onde overhead supera benefícios, desperdiçando 91% do tempo.

Table 2. Escalabilidade forte (135 imagens, GPU)			
Threads	Tempo (s)	Speedup	Eficiência (%)
1 (Sequencial)	6.91	1.00×	100.0
2	4.18	1.65×	82.6
4	2.50	2.77×	69.2
8	2.26	3.06×	38.2
32	2.40	2.88×	9.0

4.2. Escalabilidade Fraca

Aumentando proporcionalmente imagens e threads:

Table 3. Escalabilidade fraca (GPU)			
Imagens	Threads	Tempo (s)	Eficiência (%)
20	1	0.74	100.0
40	2	0.57	128.3
80	4	1.00	73.2
160	8	1.80	40.8

Observou-se eficiência super-linear de 128% com 2 threads, explicada por *pipeline* ótimo e pela melhor utilização da cache. Contudo, a eficiência piorou drasticamente para 40% com 8 threads devido a: (1) Contenção de VRAM (40MB de tensores simultâneos); (2) Fila CUDA congestionada (40% do tempo gerenciando fila); (3) GIL do Python (40% aguardando GIL + 40% aguardando GPU = 20% processamento útil).

5. Banco de Dados e Back-End

O *back-end* do sistema foi desenvolvido como uma API REST, responsável por gerenciar autenticação, armazenamento de dados e operações relacionadas aos usuários e aos carros coletados. Para isso, adotou-se uma stack tecnológica leve e eficiente, composta por Node.js para a execução do JavaScript no servidor e Express 4.18 como framework para criação das rotas HTTP e estruturação da API. O banco de dados utilizado é o SQLite, acessado por meio da biblioteca *better-sqlite3*, escolhida pela sua simplicidade, portabilidade e desempenho em projetos de pequeno porte. Para garantir segurança, utilizamos *jsonwebtoken 9.0* na geração e validação de tokens JWT, além da biblioteca *bcrypt 5.1* para criptografia de senhas. Por fim, o upload de arquivos é realizado com o auxílio da biblioteca *multer 1.4*, que permite salvar as fotos diretamente em diretórios do servidor.

A estrutura do banco de dados consiste em duas tabelas principais: *Users* e *Collected_cars*. A tabela *Users* armazena informações essenciais de cada usuário, incluindo *id* (chave primária), nome, email e senha criptografada. Já a tabela *Collected_cars* contém o registro de cada foto enviada, composta por *id* (chave primária), *id* do usuário (chave estrangeira), nome da marca, nome do modelo e o caminho da imagem no servidor. Essa arquitetura garante a organização dos dados e permite que diferentes usuários armazenem suas próprias fotos de maneira isolada e segura, possibilitando a construção de coleções individuais de carros fotografados.

6. Conclusão

Este trabalho apresentou o CARdex, um sistema inteligente voltado para a identificação automática de veículos e gamificação da prática de *car spotting*. A integração entre técnicas modernas de visão computacional, uma arquitetura distribuída e uma interface web responsiva permitiu a construção de uma plataforma capaz de reconhecer marca e modelo de automóveis com boa acurácia, ao mesmo tempo em que oferece ao usuário uma experiência organizada e engajadora. A abordagem em dois estágios de classificação, associada ao uso das arquiteturas *EfficientNet-B3* e *EfficientNet-B0*, demonstrou ser eficaz para lidar com a elevada quantidade de classes e a variabilidade visual intrínseca ao domínio automotivo.

Além do componente de IA, o sistema mostrou-se funcional como solução completa, contemplando fluxo de comunicação robusto entre front-end, back-end, servidor de inferência e banco de dados. Os testes de desempenho revelaram ganhos significativos com estratégias de paralelização por *batch* e *multi-threading*, validando a viabilidade de escalabilidade do sistema. Por fim, o CARdex abre caminho para a expansão de funcionalidades, como elementos sociais, ranking entre usuários, conquistas e melhorias no pipeline de classificação, consolidando-se como uma base promissora para aplicações futuras que combinam computação visual, gamificação e interação social estruturada.

References

- Huang, J., Chen, B., Luo, L., Yue, S., and Ounis, I. (2022). Dvm-car: A large-scale automotive dataset for visual marketing research and applications. In *Proceedings of the IEEE International Conference on Big Data (Big Data)*, pages 4130–4137. IEEE.
- Lyu, Y., Schiopu, I., Cornelis, B., and Munteanu, A. (2022). Framework for vehicle make and model recognition—a new large-scale dataset and an efficient two-branch–two-stage deep learning architecture. *Sensors*, 22(21):8439.
- Paszke, A. et al. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*.
- Zheng, Z., Wang, Y., Qian, X., Zhong, Z., Wang, Z., and Zheng, L. (2024). More’24 multimedia object re-id: Advancements, challenges, and opportunities. In *Proceedings of the International Conference on Multimedia Retrieval (ICMR)*, pages 1336–1338, Phuket, Thailand. Association for Computing Machinery.